bstorm TRAINING

Angular

Édition 2026 (Angular 22)

Construire des applications web avec Angular

Composants, signals, formulaires, routing et HttpClient

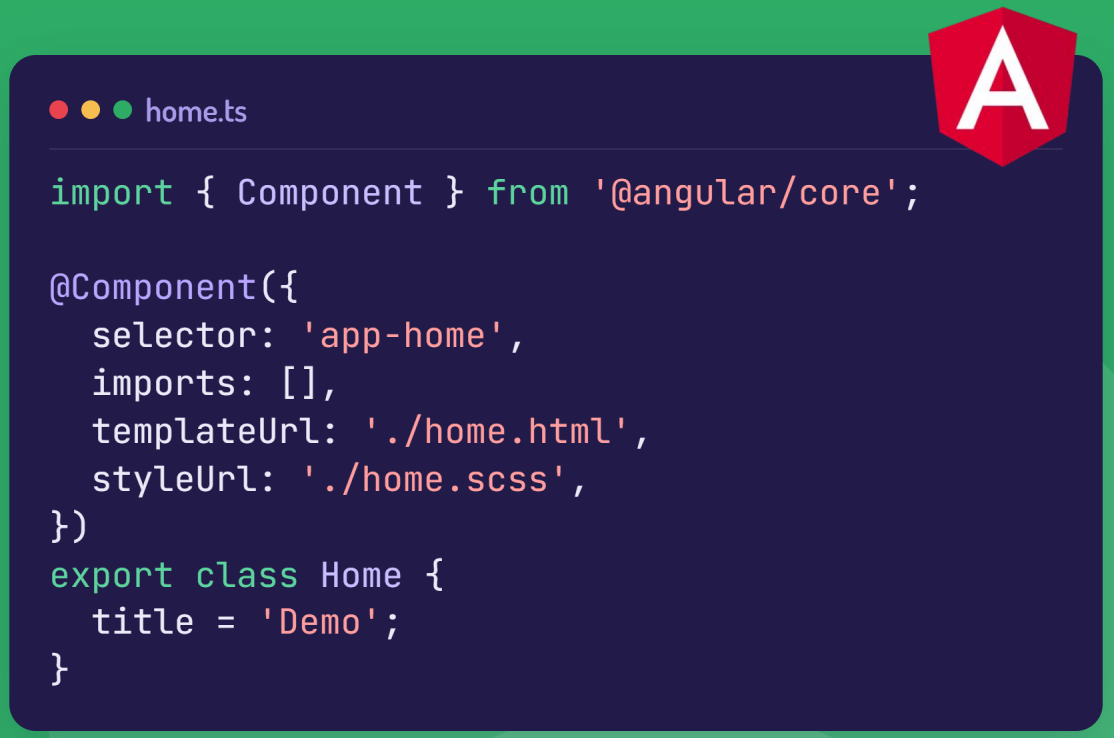


Table des matières

1 Introduction

2 Projet Angular

3 Bindings

4 Pipes

5 Control flow et directives

6 Communication entre composants

7 Signals

8 Services et injection

9 Formulaires

10 Routing

11 Storage

12 Observables et RxJS

13 HttpClient

14 Publication et mises à jour

[icon: Flag]

Le fil rouge de la semaine

ShopAdmin

Le back-office d'une petite boutique en ligne, construit étape par étape pendant les 5 jours : catalogue, panier, formulaires, clients et commandes, API.

Jour 1

Mise en place, fiche produit, pipes

Chapitres 1 à 4

Jour 2

Catalogue, directives, composants réutilisables

Chapitres 5 et 6

Jour 3

Panier en signals, services

Chapitres 7 et 8

Jour 4

Formulaires produit, navigation, guards

Chapitres 9 et 10

Jour 5

Panier persistant, recherche, API, clients, build

Chapitres 11 à 14

Exercice du chapitre — Exercice bonus sur un concept — Étape du fil rouge

Chaque matin, une branche Git contient la correction de la veille (j2-depart, j3-depart, ...). Un stagiaire en retard repart de cette branche.

01

Introduction

Angular en 2026

Introduction

1.1 Présentation d'Angular

1.2 Évolutions depuis 2021

1.3 Avantages / Inconvénients

1.4 Node.js

1.5 TypeScript

1.6 Installation

1.7 Outils

Présentation d'Angular

https://angular.dev

Angular est un framework front-end open source maintenu par Google et écrit en TypeScript.

Il permet de construire des SPA (Single Page Application). La navigation entre les pages se fait sans rechargement complet, seul le contenu nécessaire est mis à jour.

Une application Angular est un arbre de composants. Chaque composant regroupe un template HTML, des styles et une classe TypeScript.

> Une version majeure sort tous les 6 mois. Ce support est basé sur **Angular 22**.

Évolutions depuis 2021

Angular 11 (2021)		Angular 22 (2026)
NgModule obligatoire (AppModule)	→	**Composants standalone par défaut**
*ngIf, *ngFor, *ngSwitch	→	**@if, @for, @switch, @defer**
@Input() / @Output()	→	**input(), output(), model()**
Injection par constructeur	→	**inject()**
Zone.js et détection globale	→	**Signals, zoneless (défaut depuis la v21), OnPush (défaut en v22)**
Guards, resolvers et interceptors en classes	→	**Fonctions (CanActivateFn, ResolveFn, HttpInterceptorFn)**
Reactive Forms	→	**Signal Forms (stables en v22), Reactive Forms toujours supportés**
Webpack, Karma, TSLint	→	**esbuild + Vite, Vitest, ESLint**
angular.io	→	**angular.dev**

Avantages / Inconvénients

Pour
les points forts

- Framework complet (routing, formulaires, HTTP, tests)
- TypeScript natif
- Injection de dépendances intégrée
- Signals pour une mise à jour ciblée de l'écran
- CLI complet (génération, build, migrations avec ng update)
- Release tous les 6 mois et support de 18 mois par version

Contre
les points d'attention

- Courbe d'apprentissage importante
- Plusieurs syntaxes coexistent (NgModules / standalone, RxJS / signals, Reactive / Signal Forms)
- Montées de version à planifier tous les 6 mois
- Bundle initial plus lourd que des librairies plus légères

Node.js

https://nodejs.org

Node.js est un environnement d'exécution JavaScript côté serveur, basé sur le moteur V8 de Chrome.

Angular n'en a pas besoin dans le navigateur. Node.js fait tourner les outils de développement, c'est-à-dire le CLI, le serveur de développement, le build et les tests.

npm, installé avec Node.js, gère les dépendances du projet (fichier package.json).

> Angular 22 ne supporte plus Node.js 20. Installer une version LTS récente.

TypeScript

https://www.typescriptlang.org

TypeScript est un langage développé par Microsoft. C'est une surcouche de JavaScript. Tout code JavaScript est du TypeScript valide.

Il ajoute le typage statique, les interfaces, les génériques et les décorateurs. Les erreurs sont détectées à la compilation, avant l'exécution.

Le code est compilé en JavaScript pour le navigateur. Angular 22 utilise TypeScript 6.

Installation d'Angular

- Installer Node.js (version LTS) depuis le site officiel
- Installer le CLI Angular en ligne de commande
- Vérifier l'installation

terminal

```
npm install -g @angular/cli
ng version
```

terminal

```
npx -p @angular/cli<@version> ng new <name>
npx ng version
```

Le paramètre -g installe le CLI pour tout le système et pas uniquement dans le dossier courant.

Sous Windows, si PowerShell bloque la commande ng, exécuter : Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

Outils

L'éditeur utilisé dans le cadre du cours est Visual Studio Code (code.visualstudio.com). Les outils recommandés sont les suivants :

Angular Language Service

(extension VS Code) pour l'autocomplétion, les erreurs de template et la navigation dans le code

Angular DevTools

(extension Chrome et Firefox) pour inspecter l'arbre des composants, les signaux et les performances

Fichiers de consignes IA

générés par le CLI (Claude, Copilot, Cursor, Gemini) avec les bonnes pratiques Angular

angular.dev

pour la documentation, les tutoriels et le playground en ligne

02

Projet Angular

Mise en place

Projet Angular

2.1 Création du projet

2.2 Arborescence

2.3 Démarrage de l'application

2.4 Les composants

2.5 Cycle de vie des composants

2.6 Introduction à la navigation

Création du projet

La commande ng new crée un dossier contenant tout ce qui est nécessaire au framework. Options utiles :

Option	Effet
--style=tailwind	Format des feuilles de style (css, scss, sass, less, tailwind)
--ssr / --no-ssr	Active ou non le rendu côté serveur (SSR) et le prérendu
--skip-tests	Ne génère pas les fichiers de test .spec.ts
--skip-git	N'initialise pas de dépôt Git
--ai-config=none	Fichiers de consignes IA et config du serveur MCP Angular (claude-code, cursor, gemini-cli, open-ai-codex, vscode ou none)

terminal

```
ng new demo-angular --style=tailwind --no-ssr
```

Questions posées par le CLI

Sans options, le CLI pose quelques questions :

- Format des feuilles de style => choisir **SCSS** (compatible avec la plupart des librairies graphiques)
- Activer le SSR et le prérendu (SSG) => répondre **non** pour ce cours
- Outils IA à configurer (liste à cocher : Claude Code, Cursor, Gemini CLI, Codex, VS Code) => Entrée sans rien cocher pour aucun

> Ces questions peuvent être évitées en ajoutant les options correspondantes (--style=scss --no-ssr --ai-config=none).

> Le routing est inclus d'office. Les applications générées sont **standalone** et **zoneless** par défaut.

Démarrer le serveur (ng serve)

La commande ng serve compile l'application et démarre un serveur de développement sur le port 4200 (par défaut).

- --port=4201 change le port, utile si plusieurs applications tournent en parallèle
- --open (ou -o) ouvre l'application dans le navigateur une fois compilée
- Le build utilise esbuild et Vite. Chaque sauvegarde recompile l'application et met le navigateur à jour sans relancer le serveur

terminal

```
ng serve --open
ng serve --port=4201
ng serve -o
```

Arborescence du projet

Fichier / dossier	Utilité
/node_modules	Dépendances installées par npm (ne pas modifier)
/public	Fichiers statiques copiés tels quels (images, favicon). Remplace /assets
/src	Code source de l'application. C'est là que l'on travaille
angular.json	Configuration du workspace et du build
package.json	Dépendances et scripts npm du projet
tsconfig.json	Configuration du compilateur TypeScript
.vscode	Extensions et configuration recommandées pour VS Code

Les dossiers /e2e et /environments et le fichier tslint.json ne sont plus générés. ng generate environments recrée les fichiers d'environnement.

Arborescence du dossier /src

Fichier / dossier	Utilité
/app	Composants, services et routes de l'application
app.ts / app.html / app.scss	Composant racine de l'application
app.config.ts	Configuration globale (providers : routing, HTTP, ...)
app.routes.ts	Définition des routes
index.html	Page d'entrée qui contient la balise <app-root>
main.ts	Point de démarrage : bootstrapApplication(App, appConfig)
styles.scss	Styles globaux de l'application

Depuis Angular 20, les fichiers générés n'ont plus de suffixe : app.ts et non app.component.ts, classe App et non AppComponent.

Démarrage de l'application

Depuis Angular 17, un nouveau projet n'a plus de module racine (AppModule).

L'application démarre directement sur un composant standalone.

main.ts appelle bootstrapApplication() avec le composant racine et la configuration.

> app.config.ts regroupe les providers globaux (routing, HTTP, gestion des erreurs).

main.ts

```ts
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';
bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

app.config.ts

```ts
import { ApplicationConfig,
  provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
  ],
};
```

Les NgModules dans les projets existants

Les NgModules (@NgModule) restent supportés et se rencontrent dans la plupart des projets existants. Un module déclare :

- **declarations** : les composants, directives et pipes du module
- **imports** : les modules utilisés par le module
- **exports** : ce que le module rend disponible aux autres modules
- **providers** : les services fournis par le module
- **bootstrap** : le composant de démarrage (AppModule uniquement)

> Pour un nouveau développement, les composants standalone sont la norme. Un projet existant peut être migré avec la commande suivante :

terminal

```
ng generate @angular/core:standalone
```

Les composants

Le composant est l'unité de travail en Angular. Chaque page et chaque élément d'interface est un composant composé de :

- **home.ts** => classe TypeScript (logique du composant)
- **home.html** => template visuel
- **home.scss** => feuille de style, isolée des autres composants
- **home.spec.ts** => tests unitaires (Vitest par défaut depuis Angular 21)

terminal

```
ng generate component components/home
ng g c components/about
```

Composant => TS

home.ts

```ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-home',
  imports: [],
  templateUrl: './home.html',
  styleUrl: './home.scss',
})
export class Home {
  title = 'Demo';
}
```

Le décorateur @Component décrit le composant :

- **selector** : balise qui permet d'afficher le composant dans un autre template
- **imports** : composants, directives et pipes utilisés dans le template
- **templateUrl** ou **template** : template HTML (fichier séparé ou inline)
- **styleUrl** ou **styles** : styles du composant

La classe contient la logique (propriétés, méthodes).

> Depuis Angular 22, la détection des changements OnPush est appliquée par défaut aux nouveaux composants.

Appel par sélecteur

Pour afficher un composant dans le template d'un autre :

- Ajouter le composant au tableau imports du parent
- Utiliser son sélecteur dans le template du parent

> Les balises auto-fermantes sont acceptées (<app-about />).

home.ts

```ts
import { Component } from '@angular/core';
import { About } from '../about/about';

@Component({
  selector: 'app-home',
  imports: [About],
  templateUrl: './home.html',
})
export class Home {}
```

home.html

```html
<h1>Accueil</h1>
<app-about />
```

Composant => HTML

Le fichier .html d'un composant contient du HTML ainsi que la syntaxe propre à Angular :

`{{ }}`

Interpolation {{ }} et bindings

`@if`

Blocs de contrôle @if, @for, @switch, @defer

`|`

Pipes

`[ ]`

Directives

> Chaque élément est détaillé dans les chapitres suivants.

Exercice bonus · Composants

Pratiquer le concept

≈ 15 min

1. Générer trois composants header, sidebar et footer avec ng g c
2. Les afficher dans App par leur sélecteur pour former un layout
3. Passer le footer en template inline (template au lieu de templateUrl)
4. Ajouter une propriété title dans le header et l'afficher avec {{ }}

Bloqué ? Un indice vous attend en fin de support (slide 160).

Cycle de vie des composants

Le cycle de vie représente les différents moments de l'existence d'un composant. Angular appelle une méthode (hook) à chaque étape. Les plus utilisés sont les suivants :

Hook	Moment d'exécution
constructor	Création de la classe (injection uniquement, les inputs ne sont pas encore disponibles)
ngOnInit	Une seule fois, après l'initialisation des inputs
ngOnChanges	À chaque changement de valeur d'un input
ngAfterViewInit	Après le premier rendu de la vue du composant
ngOnDestroy	Juste avant la destruction du composant (libération des ressources)

angular.dev/guide/components/lifecycle

Hooks en pratique

Hooks classiques

home.ts

```ts
import { Component, OnInit, OnDestroy }
from '@angular/core';

export class Home implements OnInit, OnDestroy {
  ngOnInit(): void {
    console.log('Composant initialisé');
  }

  ngOnDestroy(): void {
    console.log('Composant détruit');
  }
}
```

La classe implémente l'interface du hook (OnInit, OnDestroy) et définit la méthode correspondante.

Alternatives fonctionnelles

home.ts

```ts
export class Home {
  constructor() {
    inject(DestroyRef).onDestroy(() => {
      console.log('Composant détruit');
    });
    afterNextRender(() => {
      // accès au DOM après le rendu
    });
  }
}
```

- DestroyRef.onDestroy() et takeUntilDestroyed() libèrent les ressources
- effect() réagit au changement d'un signal (chapitre 7)
- afterNextRender() et afterEveryRender() donnent accès au DOM

Les bases de la navigation

app.routes.ts

```ts
import { Routes } from '@angular/router';
import { Home } from './components/home/home';
import { About } from './components/about/about';
export const routes: Routes = [
  { path: '', redirectTo: 'home', pathMatch: 'full' },
  { path: 'home', component: Home },
  { path: 'about', component: About },
];
```

app.html

```html
<nav>
  <a routerLink="/home">Accueil</a>
  <a routerLink="/about">À propos</a>
</nav>
<router-outlet />
```

Les routes associent une URL à un composant. **routerLink** charge le composant à l'emplacement de <router-outlet />.

> L'attribut href recharge toute la page, ce qui va à l'encontre du principe de Single Page Application.

app.ts

```ts
@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink],
  templateUrl: './app.html',
})
export class App {}
```

> Sans **RouterLink** dans imports, le lien ne fait rien, sans aucune erreur affichée.

Routes enfants

app.routes.ts

```ts
export const routes: Routes = [
  {
    path: 'home',
    component: Home,
    children: [
      { path: 'demo', component: Demo },
    ],
  },
];
```

La propriété children définit des routes enfants, comparables à une structure dossier / sous-dossier.

Le composant parent (Home) doit contenir son propre <router-outlet />.

> L'exemple donne lieu à l'URL http://localhost:4200/home/demo

Lazy loading

app.routes.ts

```ts
export const routes: Routes = [
  {
    path: 'demo',
    loadComponent: () =>
      import('./demo/demo').then((m) => m.Demo),
  },
  {
    path: 'produits',
    loadChildren: () =>
      import('./produits/produits.routes')
        .then((m) => m.PRODUITS_ROUTES),
  },
];
```

- **loadComponent** charge un composant à la demande
- **loadChildren** charge un fichier de routes complet (une fonctionnalité)

produits/produits.routes.ts

```ts
export const PRODUITS_ROUTES: Routes = [
  { path: 'liste', component: Liste },
];
```

> Chemins **relatifs** : on écrit **liste**, pas /produits/liste. L'URL finale est /produits/liste.

Le code n'est téléchargé qu'à la première visite de la route, ce qui réduit le chargement initial.

Comment se construit une URL

`localhost:4200/` **`produits`** `/` **`liste`**

`produits` vient de app.routes.ts

app.routes.ts

```
{
  path: 'produits',
  loadChildren: () ⇒ import(...)
}
```

`liste` vient de produits.routes.ts

produits/produits.routes.ts

```
{
  path: 'liste',
  component: Liste
}
```

children

Les routes enfants sont écrites dans le même fichier que le parent et chargées avec lui.

loadChildren

Les routes enfants sont dans un fichier à part, téléchargé à la première visite.

Mettre en place un layout

app.html

```html
<div class="layout">
  <app-nav />
  <main>
    <router-outlet />
  </main>
</div>
```

app.scss

```scss
.layout { display: flex; min-height: 100vh; }
app-nav { width: 240px; }
main { flex: 1; padding: 24px; }
```

nav.html

```html
<nav>
  <a routerLink="/home">Accueil</a>
  <a routerLink="/produits/liste">Produits</a>
</nav>
```

- Le menu est un composant à part (**Nav**). Il importe **RouterLink**.
- **App** importe **Nav** et **RouterOutlet**.
- **display: flex** place le menu et le contenu côte à côte.

> Le menu reste affiché en permanence. Seul le contenu de <router-outlet /> change.

Exercice bonus · Cycle de vie et navigation

Pratiquer le concept

≈ 15 min

1. Créer deux composants Page A et Page B accessibles par deux routes et un menu routerLink
2. Dans chacun, loguer ngOnInit et ngOnDestroy, puis observer la console en naviguant
3. Charger Page B avec loadComponent, et Page A avec loadChildren (Page A a son propre fichier de routes)
4. Ajouter une route enfant /page-a/detail avec son propre <router-outlet />

Bloqué ? Un indice vous attend en fin de support (slide 161).

À VOUS DE JOUER

Exercice

- **Home**, chargée avec **loadComponent**, sur /home
- **Exercices**, chargée avec **loadChildren** et son propre fichier de routes. Elle affiche trois exercices en **routes enfants** : /exercices/exo-1, exo-2, exo-3
- Un composant de navigation à gauche, inclus dans le composant racine. « Exercices » y est un **menu déroulant** qui liste les trois exercices
- L'URL vide redirige vers /home

Astuce HTML : les balises <details> et <summary> créent un menu déroulant sans code.

Slides utiles : 22 à 24, 29 à 33. Bloqué ? Indices en fin de support (slides 162 et 163).

Arborescence attendue

```
src/app/
  app.routes.ts
  home/
  nav/
  exercices/
    exercices.ts
    exercices.routes.ts
    exo-1/ exo-2/ exo-3/
```

[icon: Flag]

Fil rouge ShopAdmin · Jour 1 · étape 1 / 13

Mise en place du projet

- ng new shop-admin (SCSS, sans SSR)
- Layout : header, menu latéral et zone <router-outlet />
- Routes en lazy loading : /catalogue, /panier et /clients, avec redirection de '' vers /catalogue
- Composants de page vides : CataloguePage, CartPage, CustomersPage

Livrable

On navigue entre les trois pages depuis le menu, sans rechargement.

03

Bindings

Lier la classe et le template

Bindings

3.1 Interpolation et property binding

3.2 Event binding

3.3 Two-way binding

3.4 Attribute, class et style binding

Interpolation et property binding

Le binding relie les propriétés de la classe TypeScript au template :

- **{{ }}** (interpolation) affiche une valeur dans le texte
- **[propriété]** lie une propriété d'un élément HTML ou d'un composant

> Un signal se lit en l'appelant : isDisabled()

Une propriété private n'est pas accessible dans le template. protected est conseillé pour les membres utilisés uniquement par le template.

demo1.ts

```ts
export class Demo1 {
  message = 'World';
  imageUrl = 'logo.png';
  isDisabled = signal(true);
}
```

demo1.html

```html
<p>Hello {{ message }}</p>
<img [src]="imageUrl" alt="logo" />
<button [disabled]="isDisabled()">
  Valider
</button>
```

Event binding

L'event binding lie un événement du template (click, input, keyup, focus, ...) à une méthode de la classe.

- **(click)="methode()"** appelle la méthode
- Des paramètres peuvent être passés à la méthode
- **$event** donne accès à l'objet événement
- **(keyup.enter)** filtre sur une touche

demo2.ts

```ts
export class Demo2 {
  count = signal(0);

  increment(step: number) {
    this.count.update((c) => c + step);
  }

  onKey(event: Event) {
    console.log((event.target as HTMLInputElement).value);
  }
}
```

demo2.html

```html
<p>Compteur : {{ count() }}</p>
<button (click)="increment(1)">+1</button>
<button (click)="increment(10)">+10</button>
<input (keyup.enter)="onKey($event)" />
```

Two-way binding

La syntaxe [( )] combine property binding et event binding. Avec ngModel, elle lie un champ de formulaire à une propriété dans les deux sens.

Si la valeur est modifiée (dans le TS ou dans l'input), la mise à jour se fait des deux côtés sans recharger la page.

ngModel accepte un signal.

> FormsModule doit être importé dans le composant qui utilise [(ngModel)].

demo3.ts

```ts
import { FormsModule } from '@angular/forms';
@Component({
  selector: 'app-demo3',
  imports: [FormsModule],
  templateUrl: './demo3.html',
})
export class Demo3 {
  name = signal('World');
}
```

demo3.html

```html
<input [(ngModel)]="name" />
<p>Hello {{ name() }}</p>
```

Attribute, class et style binding

- **[attr.x]** pour les attributs HTML sans propriété DOM (colspan, aria-*)
- **[class.x]** ajoute une classe selon une expression booléenne
- **[class]** accepte un objet de classes
- **[style.x]** applique un style inline, avec unité optionnelle (.px, .%)

> Ces bindings remplacent les directives NgClass et NgStyle, qui ne sont plus recommandées.

demo4.html

```
<button [disabled]="!isValid()">Envoyer</button>

<td [attr.colspan]="colspan">...</td>

<div [class.active]="isActive()">...</div>
<div [class]="{ active: isActive(),
                error: hasError() }">...</div>

<p [style.color]="color()">Texte</p>
<p [style.font-size.px]="size()">Texte</p>
```

Exercice bonus · Bindings

Pratiquer le concept

≈ 15 min

1. Afficher une fiche (nom, prix, image) avec {{ }} et [src]
2. Un bouton « J'aime » qui incrémente un compteur avec (click)
3. Un champ quantité lié en [(ngModel)] avec le total affiché en direct
4. [class.epuise] quand le stock vaut 0 et [style.width.%] pour une barre de stock

À VOUS DE JOUER

Exercice

Dans un module exo-2 :

- un input button qui passe de On à Off au click (value). un second bouton qui passe disabled quand le premier bouton est à Off.
- un input button qui compte les clics (value).
- un input text et ca valeur affichée en temps réel dans un span.

[icon: Flag]

Fil rouge ShopAdmin · Jour 1 · étape 2 / 13

Fiche produit

- Interface Product : id, name, price, stock, category, imageUrl
- CataloguePage affiche un produit en dur : nom, prix et image avec [src]
- Bouton « Ajouter au panier » qui incrémente un compteur affiché dans la page
- [class.out-of-stock] quand le stock vaut 0, et bouton désactivé avec [disabled]

Livrable

Une fiche produit interactive dans le catalogue.

04

Pipes

Formater l'affichage

Les pipes

4.1 Fonctionnement

4.2 Les principaux pipes

4.3 Pipe personnalisé

Les pipes, fonctionnement

Un pipe transforme l'affichage d'une valeur dans le template, sans modifier la valeur d'origine.

Le nom vient du caractère | utilisé pour l'appeler. Les pipes acceptent des paramètres (après :) et peuvent s'enchaîner.

> Les pipes intégrés (UpperCasePipe, DatePipe, CurrencyPipe, ...) doivent être ajoutés aux imports du composant.

TypeScript

```ts
message = ' + Salut les gars + ';
today = new Date();
price = 1250.5;
```

HTML

```html
<p>{{ message | uppercase }}</p>
<p>{{ today | date: ' + dd/MM/yyyy + ' }}</p>
<p>{{ price | currency: ' + EUR + ' }}</p>
```

NAVIGATEUR

```
SALUT LES GARS
08/10/2026
€1,250.50
```

Les principaux pipes

Pipe	Utilité
| date: 'format'	Formate l'affichage d'une date
| uppercase / lowercase	Transforme la chaîne en majuscules / minuscules
| titlecase	Première lettre de chaque mot en majuscule
| currency: 'EUR'	Affiche un montant dans la devise donnée
| number: '1.0-2'	Formate un nombre (décimales minimum et maximum)
| percent	Affiche un pourcentage
| json	Convertit la valeur en JSON (utile pour le debug)
| async	S'abonne à un Observable ou une Promise et affiche sa valeur

Affichage belge (1 250,50 €) : locale fr-BE et LOCALE_ID dans app.config.ts. Liste complète sur angular.dev.

Pipe personnalisé

terminal

```
ng generate pipe pipes/to-fahrenheit
```

to-fahrenheit pipe

```
import { Pipe, PipeTransform } from '@angular/core';
@Pipe({ name: 'toFahrenheit' })
export class ToFahrenheitPipe implements PipeTransform {
  transform(value: number, decimals = 1): string {
    const f = (value * 9) / 5 + 32;
    return f.toFixed(decimals) + ' °F';
  }
}
```

Un pipe personnalisé répond à un besoin précis (conversion, format d'affichage spécifique).

- **@Pipe** définit le nom utilisé dans le template
- La classe implémente l'interface **PipeTransform**
- **transform()** reçoit la valeur, puis les paramètres éventuels
- Le type de retour est le type affiché

Utiliser un pipe personnalisé

Le pipe est standalone. Il suffit de l'ajouter aux imports du composant qui l'utilise.

Le SharedModule utilisé dans les anciennes versions pour partager les pipes entre modules n'est plus nécessaire.

> Un pipe est pur par défaut : il n'est recalculé que si sa valeur d'entrée change.

Pour un calcul utilisé uniquement dans un composant, un computed() est souvent plus simple (chapitre 7).

demo5.ts

```ts
import { ToFahrenheitPipe }
  from '../pipes/to-fahrenheit-pipe';

@Component({
  selector: 'app-demo5',
  imports: [ToFahrenheitPipe],
  template: `
    <p>{{ 21 | toFahrenheit }}</p>
    <p>{{ 21 | toFahrenheit: 2 }}</p>
  `,
})
export class Demo5 {}
```

Exercice bonus · Pipes

Pratiquer le concept

≈ 15 min

1. Afficher une date de commande avec trois formats du pipe date
2. Afficher un prix avec currency en EUR puis en USD, et un taux avec percent
3. Créer un pipe truncate qui coupe un texte après n caractères et ajoute « … » (20 par défaut)
4. Enchaîner les pipes : {{ description | truncate: 30 | uppercase }}

À VOUS DE JOUER

Exercice

Mettre en place un chronomètre :

- 3 boutons Start, Pause et Reset, actifs ou inactifs en fonction de l'état du chronomètre
- Affichage « xx minutes xx secondes » à l'aide d'un pipe personnalisé toTime
- Le temps écoulé est stocké dans un signal

Astuce : setInterval() et clearInterval(), ainsi que l'interpolation de chaînes TypeScript (template literals).

[icon: Flag]

Fil rouge ShopAdmin · Jour 1 · étape 3 / 13

Pipes du catalogue

- Prix affichés avec currency: 'EUR'
- Pipe personnalisé stockLabel : « En stock », « Plus que n » (stock ≤ 3), « Rupture »
- Nom du produit en titlecase

Livrable

Fin du jour 1 : projet, routes, fiche produit et pipe stockLabel. C'est la base du jour 2.

05

Control flow et directives

Structurer le template

Control flow et directives

5.1 @if

5.2 @for

5.3 @switch et @let

5.4 @defer

5.5 Les directives

5.6 Directive personnalisée

Blocs de contrôle : @if

Depuis Angular 17, le template intègre des blocs de contrôle. Ils remplacent *ngIf, *ngFor et *ngSwitch (dépréciés depuis Angular 20).

- Aucun import n'est nécessaire
- @else if et @else sont optionnels
- **as** crée un alias de la valeur testée

Migration d'un projet existant

terminal

```
ng generate @angular/core:control-flow
```

HTML

```
@if (user(); as u) {
  <p>Bonjour {{ u.name }}</p>
} @else if (loading()) {
  <p>Chargement...</p>
} @else {
  <p>Veuillez vous connecter</p>
}
```

Blocs de contrôle : @for

@for répète un bloc pour chaque élément d'une collection.

- **track** est obligatoire : identifiant unique de l'élément ($index à défaut)
- Variables disponibles : $index, $first, $last, $even, $odd, $count
- **@empty** s'affiche si la collection est vide

HTML

```
<ul>
  @for (item of items(); track item.id;
        let i = $index) {
    <li>{{ i + 1 }}. {{ item.name }}</li>
  } @empty {
    <li>Aucun élément</li>
  }
</ul>
```

Blocs de contrôle : @switch et @let

@switch affiche un bloc selon la valeur d'une expression. @default est optionnel.

@let déclare une variable locale dans le template. Elle est recalculée à chaque changement de ses dépendances.

@switch

```
@switch (role()) {
  @case ('admin') { <app-admin-panel /> }
  @case ('user') { <app-dashboard /> }
  @default { <p>Accès refusé</p> }
}
```

@let

```
@let total = price() * quantity();
<p>Total : {{ total }} €</p>
```

Chargement différé : @defer

@defer charge une partie du template (et le code des composants qu'elle contient) uniquement au moment voulu. Déclencheurs disponibles :

- **on idle** (par défaut) : quand le navigateur est inactif
- **on viewport** : quand la zone devient visible
- **on interaction**, **on hover**, **on timer(2s)**
- **when** condition : quand une expression devient vraie

HTML

```
@defer (on viewport) {
  <app-heavy-chart />
} @placeholder {
  <p>Le graphique se charge au défilement</p>
} @loading (minimum 500ms) {
  <p>Chargement...</p>
} @error {
  <p>Erreur de chargement</p>
}
```

Exercice bonus · Control flow

Pratiquer le concept

≈ 15 min

1. Liste de tâches avec @for, track et @empty. $index pour numéroter, $even pour alterner la couleur
2. @if / @else : afficher « Tout est fait » quand plus aucune tâche n'est à faire
3. @switch sur la priorité (haute, moyenne, basse) pour afficher une pastille
4. @defer (on viewport) sur un bloc de statistiques en bas de page, avec @placeholder

Les directives

Une directive est une classe qui ajoute un comportement à un élément du template. Il en existe trois types :

Type	Description	Exemples
Composants	Directives avec un template	<app-home />
Directives d'attribut	Modifient l'apparence ou le comportement d'un élément	ngModel, routerLink, routerLinkActive, directives personnalisées
Directives structurelles	Ajoutent ou retirent des éléments du DOM	*ngIf, *ngFor (remplacées par @if et @for)

NgClass et NgStyle fonctionnent toujours mais les bindings [class] et [style] sont recommandés. Les migrations ngclass-to-class et ngstyle-to-style convertissent un projet existant.

Directive personnalisée

highlight.ts

```ts
import { Directive, ElementRef, inject, input }
  from '@angular/core';
@Directive({
  selector: '[appHighlight]',
  host: {
    '(mouseenter)': 'setColor(color())',
    '(mouseleave)': 'setColor(null)',
  },
})
export class Highlight {
  color = input('yellow', { alias: 'appHighlight' });
  private el = inject(ElementRef);

  setColor(color: string | null) {
    this.el.nativeElement.style.backgroundColor = color;
  }
}
```

- **host** écoute les événements de l'élément (remplace @HostListener et @HostBinding)
- **inject(ElementRef)** donne accès à l'élément
- **input()** avec alias reçoit la valeur de l'attribut

> La directive est ajoutée aux imports du composant qui l'utilise.

terminal

```
ng g directive directives/highlight
```

Utilisation

```html
<p appHighlight="lightblue">
  Survolez-moi
</p>
```

Exercice bonus · Directives

Pratiquer le concept

≈ 15 min

1. Directive appTooltip : affiche un texte (input) au survol, avec host
2. Directive appAutofocus : donne le focus à l'élément au chargement (afterNextRender)
3. Remplacer un [ngClass] existant par un binding [class]

À VOUS DE JOUER

Exercice

À partir d'une interface Link (titre, url, liens enfants), mettre en place un menu de navigation construit avec @for à partir d'une liste définie dans le composant nav.

- Chaque fonctionnalité (Home, Exercices) est représentée dans le menu avec un sous-menu affiché ou masqué au clic
- Le lien actif est mis en évidence avec routerLinkActive

[icon: Flag]

Fil rouge ShopAdmin · Jour 2 · étape 4 / 13

Catalogue complet

```
git checkout j2-depart
```

- 10 produits en dur, affichés avec @for (track product.id) et @empty
- Filtre par catégorie avec des boutons, @switch pour l'icône de chaque catégorie
- Directive appLowStock qui met la carte en évidence quand le stock est ≤ 3
- @defer (on viewport) sur les images des produits

Livrable

Un catalogue filtrable avec les stocks faibles mis en évidence.

06

Communication entre composants

input(), output() et model()

Communication entre composants

6.1 Principe

6.2 input()

6.3 output()

6.4 model()

6.5 Projection de contenu

Faire discuter les composants

Un composant parent intègre un composant enfant par son sélecteur. Ils communiquent de deux façons :

- Le parent transmet des données à l'enfant par des **inputs**
- L'enfant prévient le parent par des **outputs** (événements)

parent.html

```
<app-child [title]="pageTitle" (selected)="onSelected($event)" />
```

Depuis Angular 17, les fonctions input(), output() et model() remplacent les décorateurs @Input() et @Output(). Les décorateurs fonctionnent toujours dans les projets existants.

input()

child.ts (enfant)

```ts
import { Component, input } from '@angular/core';
@Component({
  selector: 'app-child',
  template: `
    <h2>{{ title() }}</h2>
    <p>{{ count() }} élément(s)</p>
  `,
})
export class Child {
  title = input.required<string>();
  count = input(0);
}
```

parent.html

```html
<app-child [title]="pageTitle" [count]="items().length"
/>
```

- **input()** crée un signal en lecture seule, avec une valeur par défaut
- **input.required()** rend la valeur obligatoire. Son absence provoque une erreur de compilation
- La valeur se lit en appelant le signal : title()
- L'enfant ne modifie pas un input

output()

child.ts (enfant)

```ts
import { Component, output } from '@angular/core';
@Component({
  selector: 'app-child',
  template: `<button
(click)="choose()">Choisir</button>`,
})
export class Child {
  selected = output<string>();

  choose() {
    this.selected.emit('Article 1');
  }
}
```

parent.html

```html
<app-child (selected)="onSelected($event)" />
```

- **output()** déclare un événement typé
- **emit()** envoie la valeur au parent
- Le parent récupère la valeur dans **$event**, par event binding

> output() n'est pas un signal et n'a pas besoin d'EventEmitter.

model()

model() crée un input que l'enfant peut aussi modifier.

Le parent le lie en two-way binding avec [( )]. Une modification dans l'enfant met à jour le signal du parent.

> Cas d'usage : composants de saisie réutilisables (compteur, sélecteur, toggle).

counter.ts (enfant)

```ts
export class Counter {
  value = model(0);

  increment() {
    this.value.update((v) => v + 1);
  }
}
```

parent.html (total = signal(0))

```html
<app-counter [(value)]="total" />
<p>Total : {{ total() }}</p>
```

Projection de contenu (ng-content)

Le parent peut fournir du contenu HTML à l'enfant. L'enfant l'affiche à l'emplacement de la balise <ng-content />.

> L'attribut select définit plusieurs zones de projection.

card.html (enfant)

```
<div class="card">
  <ng-content select="[card-title]" />
  <ng-content />
</div>
```

parent.html

```
<app-card>
  <h2 card-title>Titre</h2>
  <p>Contenu de la carte</p>
</app-card>
```

Exercice bonus · input(), output() et model()

Pratiquer le concept

≈ 15 min

1. Composant RatingStars : input value, output rated avec la note cliquée
2. Deuxième version avec model() pour utiliser [(value)]
3. Composant Panel avec ng-content : titre projeté avec select, contenu par défaut
4. Le parent affiche trois RatingStars et la moyenne des notes

À VOUS DE JOUER

Exercice

Mettre en place une liste de courses en deux composants :

- Le premier contient un champ texte permettant d'ajouter un article à la liste
- Le deuxième affiche la liste et permet de supprimer un article

Les deux composants sont affichés sur la même page et communiquent par input() et output().

- utilisez un interface pour les articles (nom / prix).

[icon: Flag]

Fil rouge ShopAdmin · Jour 2 · étape 5 / 13

Composants réutilisables

- ProductCard (dumb) : input product, output addToCart
- CataloguePage (smart) garde la liste et reçoit les ajouts au panier
- QuantityPicker avec model(), utilisé dans la carte
- Composant Panel avec ng-content pour les blocs de page

Livrable

Le catalogue est découpé en composants smart et dumb.

07

Signals

Réactivité et état local

Signals

7.1 Principe

7.2 signal()

7.3 computed()

7.4 effect()

7.5 linkedSignal()

7.6 resource()

Principe des signals

Un signal est une valeur qui prévient Angular quand elle change. Angular met alors à jour uniquement les parties de l'écran qui l'utilisent.

Lecture

`appel du signal, count()`

Écriture

`set() ou update()`

Valeur dérivée

`computed()`

Effet de bord

`effect()`

> Depuis Angular 21, les nouvelles applications fonctionnent sans Zone.js (zoneless). La mise à jour de l'écran repose sur les signals, les événements du template et le pipe async.

signal()

cart.ts

```ts
import { signal } from '@angular/core';
export class Cart {
  items = signal<string[]>([]);
  readonly list = this.items.asReadonly();

  add(item: string) {
    this.items.update((list) => [...list, item]);
  }

  clear() {
    this.items.set([]);
  }
}
```

- **set()** remplace la valeur
- **update()** calcule la nouvelle valeur à partir de l'ancienne
- Pour un tableau ou un objet, créer une nouvelle référence (spread) plutôt que de modifier l'existant
- **asReadonly()** expose le signal en lecture seule

computed()

cart.ts

```ts
export class Cart {
  items = signal<Item[]>([]);

  total = computed(() =>
    this.items().reduce(
      (sum, i) => sum + i.price * i.qty, 0)
  );

  isEmpty = computed(() => this.items().length === 0);
}
```

cart.html

```html
<p>Total : {{ total() | currency: 'EUR' }}</p>
```

computed() crée un signal dérivé en lecture seule.

Il est recalculé uniquement quand un signal dont il dépend change. Sa valeur est mise en cache entre deux lectures.

> computed() remplace la plupart des méthodes appelées dans le template.

effect()

settings.ts

```ts
export class Settings {
  theme = signal<'light' | 'dark'>('light');

  constructor() {
    effect(() => {
      localStorage.setItem('theme', this.theme());
    });
  }
}
```

effect() exécute du code à chaque changement des signals lus à l'intérieur.

Usage réservé aux effets de bord : sauvegarde, logs, appel à une librairie externe.

Un effect se crée dans un contexte d'injection (constructeur ou initialisation de propriété) et s'arrête à la destruction du composant.

> Pour calculer une valeur à partir d'autres signals, utiliser computed() et non effect().

Exercice bonus · signal(), computed() et effect()

Pratiquer le concept

≈ 15 min

1. Un signal contient une liste de températures. computed() pour le minimum, le maximum et la moyenne
2. update() pour ajouter une mesure, set() pour réinitialiser
3. effect() qui logue un message quand la moyenne dépasse 25
4. Un console.log dans le computed pour vérifier qu'il n'est recalculé que si la liste change

linkedSignal()

shipping-picker.ts

```ts
export class ShippingPicker {
  options = input.required<string[]>();

  selected = linkedSignal(() => this.options()[0]);

  choose(option: string) {
    this.selected.set(option);
  }
}
```

linkedSignal() crée un signal modifiable dont la valeur est recalculée quand sa source change.

Dans l'exemple, l'utilisateur peut choisir une option.

> Si la liste d'options change, la sélection revient au premier élément.

resource()

user-detail.ts

```ts
export class UserDetail {
  userId = input.required<number>();

  user = resource({
    params: () => ({ id: this.userId() }),
    loader: ({ params }) =>
      fetch(`/api/users/${params.id}`)
        .then((r) => r.json()),
  });
}
```

user-detail.html

```html
@if (user.isLoading()) { <p>Chargement...</p> }
@if (user.hasValue()) { <p>{{ user.value().name }}</p> }
```

resource() charge une donnée asynchrone à partir de signals. Quand userId change, la requête est relancée.

L'état est exposé sous forme de signals : value(), hasValue(), isLoading(), error(), status().

> httpResource() fait la même chose avec HttpClient (chapitre 13). resource() et httpResource() sont stables depuis Angular 22.

Exercice bonus · linkedSignal() et resource()

Pratiquer le concept

≈ 15 min

1. linkedSignal : liste de tailles reçue en input, sélection par défaut sur la première, réinitialisée si la liste change
2. resource() qui charge jsonplaceholder.typicode.com/users/:id selon un signal id
3. Boutons Précédent / Suivant qui changent l'id
4. Afficher isLoading(), error() et value()

À VOUS DE JOUER

Exercice

Reprendre le chronomètre de l'exercice du chapitre 4 :

- Le temps écoulé et l'état (arrêté, en cours, en pause) sont des signals
- L'état actif ou inactif de chaque bouton est un computed()
- Un effect() enregistre le meilleur temps dans le localStorage

[icon: Flag]

Fil rouge ShopAdmin · Jour 3 · étape 6 / 13

Panier en signals

```
git checkout j3-depart
```

- Signal cartItems : liste de { product, quantity }
- computed() : nombre d'articles, sous-total, TVA à 21 % et total TTC
- Ajouter, retirer et modifier une quantité avec update()
- linkedSignal : la catégorie sélectionnée revient à « Toutes » quand la liste des catégories change

Livrable

Un panier réactif dont les totaux se mettent à jour tout seuls.

08

Services et injection

Partager la logique

Les services

8.1 Rôle des services

8.2 Création d'un service

8.3 Injection avec inject()

8.4 Portée des providers

Les services, pourquoi et comment

Un service regroupe une logique réutilisable, indépendante de l'affichage :

Accès aux API (HttpClient)

État partagé entre composants (panier, utilisateur connecté)

Règles métier et calculs

Le composant reste centré sur l'affichage et délègue le reste aux services. Les services constituent le cœur de la logique fonctionnelle de l'application.

> Angular fournit les services aux composants par injection de dépendances.

Création d'un service

terminal

```
ng generate service services/shopping-list
```

shopping-list service

```
import { Injectable, signal } from '@angular/core';
@Injectable({ providedIn: 'root' })
export class ShoppingListService {
  private readonly items = signal<string[]>([]);
  readonly list = this.items.asReadonly();

  add(item: string) {
    this.items.update((l) => [...l, item]);
  }

  remove(item: string) {
    this.items.update((l) => l.filter((i) => i
=/= item));
  }
}
```

- **@Injectable** signale que la classe peut être injectée
- **providedIn: 'root'** crée une instance unique (singleton) pour toute l'application
- L'état est un signal privé, exposé en lecture seule

> Depuis Angular 22, le CLI génère le décorateur @Service(), équivalent à @Injectable({ providedIn: 'root' }). L'option --injectable conserve l'ancienne forme.

Injection avec inject()

inject() (recommandé)

```
import { Component, inject } from '@angular/core';
export class ShoppingPage {
  private readonly shopping =
inject(ShoppingListService);
  protected readonly items = this.shopping.list;

  add(item: string) {
    this.shopping.add(item);
  }
}
```

Injection par constructeur (projets existants)

```
constructor(private shopping: ShoppingListService) {}
```

inject() récupère l'instance du service. Il s'utilise à l'initialisation d'une propriété ou dans le constructeur.

L'injection par constructeur reste valable. inject() est la forme recommandée et elle est obligatoire avec @Service().

> La migration ng generate @angular/core:inject convertit un projet existant.

Portée des providers

Le provider détermine la portée et la durée de vie de l'instance du service :

Provider	Portée et durée de vie	Mise en place
Racine (singleton)	Une instance pour toute l'application, créée à la première utilisation. Retirée du build si jamais utilisée	providedIn: 'root' ou @Service()
Application	Une instance pour toute l'application, déclarée explicitement	providers de app.config.ts
Route	Une instance partagée par la route et ses enfants. Disponible une fois la route chargée (lazy loading)	providers dans la définition de la route
Composant	Une nouvelle instance par composant (et ses enfants), détruite avec lui	providers de @Component

Exercice bonus · Services

Pratiquer le concept

≈ 15 min

1. ThemeService avec un signal light / dark, partagé par le header et la page
2. NotificationService : liste de messages, add() et remove(), et un composant toast qui l'affiche
3. Fournir un CounterService dans les providers d'un composant, afficher ce composant deux fois et constater deux instances

À VOUS DE JOUER

Exercice

Optimiser la liste de courses avec un service qui gère la liste (signal) et les méthodes qui s'y appliquent.

- Ajouter la gestion des quantités (Lait x 6, Tomates x 3, ...)
- Afficher le nombre total d'articles avec un computed()

[icon: Flag]

Fil rouge ShopAdmin · Jour 3 · étape 7 / 13

Services du catalogue et du panier

- ProductService (@Service) qui détient le catalogue
- CartService : signal privé exposé avec asReadonly(), méthodes add, remove et clear
- Badge dans le header qui affiche le nombre d'articles du panier
- Page /panier qui affiche le contenu et le total

Livrable

Le panier est partagé entre le catalogue, le header et la page panier.

09

Formulaires

Signal Forms et Reactive Forms

Les formulaires

9.1 Trois approches

9.2 Modèle et form()

9.3 Liaison des champs

9.4 Validateurs

9.5 Affichage des erreurs

9.6 Règles conditionnelles

9.7 Listes de champs

9.8 Soumission

9.9 Reactive Forms

Les différents formulaires

La plupart des interactions utilisateur se font au travers de formulaires. Angular propose trois approches :

Approche	Principe	Usage
Template-driven (FormsModule)	[(ngModel)] dans le template	Formulaires très simples
Reactive Forms (ReactiveFormsModule)	FormGroup et FormControl dans la classe	Projets existants, très répandus
Signal Forms (@angular/forms/signals)	Modèle signal + form() + [formField]	Nouveaux développements, stables depuis Angular 22

> Ce chapitre présente d'abord les Signal Forms, puis les Reactive Forms que l'on retrouve dans la plupart des projets existants.

Signal Forms : modèle et form()

login.ts

```ts
import { Component, signal } from '@angular/core';
import { form, FormField } from '@angular/forms/signals';

interface Login {
  email: string;
  password: string;
}

@Component({
  selector: 'app-login',
  imports: [FormField],
  templateUrl: './login.html',
})
export class LoginPage {
  loginModel = signal<Login>({ email: '', password: '' });
  loginForm = form(this.loginModel);
}
```

- Le **modèle** est un signal qui contient les valeurs du formulaire
- **form()** crée un arbre de champs qui suit la structure du modèle
- **loginForm.email** est le champ email
- **loginForm.email()** donne son état (valeur, validité, touched, …)

Liaison des champs : [formField]

login.html

```
<form>
  <label>Email
    <input type="email" [formField]="loginForm.email" />
  </label>

  <label>Mot de passe
    <input type="password"
           [formField]="loginForm.password" />
  </label>
</form>

<p>Email saisi : {{ loginForm.email().value() }}</p>
```

La directive [formField] relie un input à un champ du formulaire.

La liaison est bidirectionnelle : une saisie met à jour le modèle et une modification du modèle met à jour l'input.

> FormField doit être ajouté aux imports du composant.

Validateurs

login.ts

```ts
import { form, required, email, minLength }
  from '@angular/forms/signals';

loginForm = form(this.loginModel, (path) => {
  required(path.email, { message: 'Email
obligatoire' });
  email(path.email, { message: 'Format email
invalide' });
  required(path.password);
  minLength(path.password, 8, {
    message: '8 caractères minimum',
  });
});
```

Validateur	Vérifie
required	Valeur non vide
email	Format x@x.x
min / max	Valeur numérique minimale / maximale
minLength / maxLength	Longueur minimale / maximale
pattern	Correspondance avec une expression régulière
minDate / maxDate	Date minimale / maximale (Angular 22)

Affichage des erreurs

login.html

```
<input type="email" [formField]="loginForm.email" />
@if (loginForm.email().touched()
     && loginForm.email().invalid()) {
  @for (error of loginForm.email().errors();
        track error.kind) {
    <p class="error">{{ error.message }}</p>
  }
}

<button type="submit" [disabled]="loginForm().invalid()">
  Connexion
</button>
```

Chaque champ expose son état sous forme de signals :

- **value()** : valeur courante
- **valid()** / **invalid()**
- **touched()** / **dirty()**
- **errors()** : liste des erreurs (kind, message)
- **pending()** : validation asynchrone en cours

> loginForm() donne l'état du formulaire complet.

Règles conditionnelles et validateur personnalisé

order.ts

```
orderForm = form(this.orderModel, (path) => {
  required(path.promoCode, {
    when: ({ valueOf }) => valueOf(path.applyDiscount),
  });
  validate(path.website, ({ value }) => {
    if (!value().startsWith('https://')) {
      return {
        kind: 'https',
        message: "L'URL doit commencer par https://",
      };
    }
    return null;
  });
});
```

- L'option **when** applique une règle selon la valeur d'un autre champ
- **validate()** définit un validateur personnalisé. Il retourne une erreur (kind, message) ou null
- **disabled()**, **readonly()** et **hidden()** acceptent aussi l'option when
- **validateAsync()** et **validateHttp()** valident côté serveur

Listes de champs : applyEach()

order.ts

```ts
type Item = { name: string; quantity: number };
function itemSchema(item: SchemaPathTree<Item>) {
  required(item.name, { message: 'Nom obligatoire' });
  min(item.quantity, 1, { message: 'Minimum 1' });
}

orderForm = form(this.orderModel, (path) => {
  required(path.title);
  applyEach(path.items, itemSchema);
});
```

Un tableau du modèle devient une liste de champs.

applyEach() applique les mêmes règles à chaque élément du tableau.

> Le schéma d'un élément est une fonction réutilisable.

Ajouter et afficher des lignes

order.ts

```
addItem() {
  this.orderModel.update((o) => ({
    ...o,
    items: [...o.items, { name: '', quantity:
1 }],
  }));
}
removeItem(index: number) {
  this.orderModel.update((o) => ({
    ...o,
    items: o.items.filter((_, i) => i !== index),
  }));
}
```

Pour ajouter ou retirer une ligne, on modifie le modèle. Le formulaire suit automatiquement.

Dans le template, @for parcourt les champs du tableau.

order.html

```
@for (item of orderForm.items; track $index) {
  <input [formField]="item.name" />
  <input type="number" [formField]="item.quantity" />
  <button type="button"
(click)="removeItem($index)">X</button>
}
```

Soumission du formulaire

contact.ts

```
contactForm = form(
  this.contactModel,
  (path) ⇒ {
    required(path.name);
    required(path.email);
  },
  {
    submission: {
      action: async (field) ⇒ {
        await this.api.save(field().value());
      },
    },
  },
);
```

- La directive **[formRoot]** (FormRoot) gère l'événement submit et ajoute novalidate
- L'action n'est exécutée que si le formulaire est valide
- L'action peut retourner une erreur serveur { kind, message }
- La fonction **submit(form, action)** soumet le formulaire depuis le code

contact.html

```
<form [formRoot]="contactForm">
  ...
  <button type="submit">Envoyer</button>
</form>
```

Exercice bonus · Signal Forms

Pratiquer le concept

≈ 20 min

1. Formulaire d'inscription : email, mot de passe et confirmation
2. Validateur personnalisé avec validate() : la confirmation doit être identique au mot de passe
3. Champ « Société » obligatoire seulement si « Professionnel » est coché (option when)
4. Soumission avec [formRoot] qui simule une erreur serveur si l'email est déjà pris

Reactive Forms (1/2)

register.ts

```ts
import { FormBuilder, ReactiveFormsModule, Validators }
  from '@angular/forms';

export class Register {
  private fb = inject(FormBuilder).nonNullable;

  registerForm = this.fb.group({
    email: ['', [Validators.required, Validators.email]],
    age: [18, Validators.min(13)],
    series: this.fb.array<string>([]),
  });

  get series() {
    return this.registerForm.controls.series;
  }
}
```

Les Reactive Forms restent très présents dans les projets existants. Ils sont typés depuis Angular 14.

- **FormBuilder** (injecté avec inject()) crée les contrôles
- **group()** => FormGroup, **control()** => FormControl, **array()** => FormArray
- **Validators.required**, email, min, max, minLength, pattern
- **ReactiveFormsModule** doit être importé dans le composant

Reactive Forms (2/2)

register.html

```
<form [formGroup]="registerForm" (ngSubmit)="save()">
  <input formControlName="email" />
  @if (registerForm.controls.email.invalid
        && registerForm.controls.email.touched) {
    <p>Email invalide</p>
  }
  <input type="number" formControlName="age" />
  <button
[disabled]="registerForm.invalid">Valider</button>
</form>
```

register.ts

```
save() {
  const value = this.registerForm.getRawValue();
  console.log(value.email, value.age);
}
```

- **[formGroup]** lie le formulaire au FormGroup
- **formControlName** lie chaque input à son contrôle
- **(ngSubmit)** soumet le formulaire
- **value** ou **getRawValue()** (inclut les champs désactivés) donne les valeurs
- Un validateur personnalisé est une fonction de type **ValidatorFn**

Exercice bonus · Reactive Forms

Pratiquer le concept

≈ 20 min

1. Refaire le formulaire d'inscription avec FormBuilder et ReactiveFormsModule
2. Un FormArray de numéros de téléphone, avec ajout et suppression
3. Un ValidatorFn personnalisé : au moins une majuscule dans le mot de passe
4. Afficher getRawValue() à la soumission

À VOUS DE JOUER

Exercice

Mettre en place un CRUD de fans de séries avec les Signal Forms.

Chaque fan est défini par son nom, sa date de naissance (interdit aux moins de 13 ans) et la liste de ses séries préférées (juste le titre).

- Aucune limite au nombre de séries à la création du profil
- La page de modification permet d'ajouter ou de supprimer des séries
- 4 composants (liste des fans, détail, création, mise à jour)
- Seule l'année est vérifiée pour l'âge des fans
- Si un champ est visible, il est obligatoirement rempli

[icon: Flag]

Fil rouge ShopAdmin · Jour 4 · étape 8 / 13

Formulaire produit

```
git checkout j4-depart
```

- Page de création en Signal Forms : nom requis, prix > 0, stock ≥ 0, catégorie
- Liste de variantes (taille, couleur) avec applyEach(), ajout et suppression de lignes
- Erreurs affichées sous chaque champ, bouton désactivé si le formulaire est invalide
- Soumission avec [formRoot] qui ajoute le produit au ProductService

Livrable

On peut créer un produit qui apparaît ensuite dans le catalogue.

10

Routing

Navigation dans l'application

Routing

10.1 Configuration des routes

10.2 Routes d'une fonctionnalité

10.3 Navigation par le code

10.4 Paramètres de route

10.5 Guards

10.6 Resolvers

Configuration des routes

app.routes.ts

```
export const routes: Routes = [
  { path: '', redirectTo: 'home', pathMatch: 'full' },
  { path: 'home', component: Home, title: 'Accueil' },
  {
    path: 'fans',
    loadChildren: () =>
      import('./fans/fans.routes').then((m) =>
m.FAN_ROUTES),
  },
  { path: '**', component: NotFound },
];
```

- **path** : chemin de la route
- **''** : chemin vide, point d'entrée de l'application
- ** : toutes les routes inconnues (page 404), à placer en dernier
- **redirectTo** : redirection vers une autre route
- **title** : titre de l'onglet du navigateur

> Les routes sont enregistrées par provideRouter(routes) dans app.config.ts.

Routes d'une fonctionnalité

fans.routes.ts

```
export const FAN_ROUTES: Routes = [
  { path: '', component: FanList },
  { path: 'create', component: FanCreate },
  { path: ':id', component: FanDetail },
  { path: ':id/edit', component: FanEdit },
];
```

Chaque fonctionnalité regroupant plusieurs composants (administration, tableau de bord, profil) dispose de son propre fichier de routes.

Le template des URL devient monsite.com/fonctionnalite/composant.

RouterModule.forRoot() et forChild() ne sont plus nécessaires.

> En lazy loading, un service fourni au niveau d'une route n'existe qu'une fois la route chargée.

Navigation par le code

fan-create.ts

```ts
export class FanCreate {
  private router = inject(Router);

  save() {
    // ... enregistrement
    this.router.navigate(['/fans', 42]);
  }

  cancel() {
    this.router.navigateByUrl('/fans');
  }
}
```

Lien avec paramètre

```html
<a [routerLink]="['/fans', fan.id]">Détail</a>
```

Le service Router permet de naviguer depuis le code (après un enregistrement, une connexion, ...).

- **navigate()** reçoit un tableau de segments
- **navigateByUrl()** reçoit une URL complète
- Des query params peuvent être ajoutés : { queryParams: { page: 2 } }

Paramètres de route

Paramètre reçu comme input

```ts
// app.config.ts
provideRouter(routes, withComponentInputBinding())

// fan-detail.ts
export class FanDetail {
  id = input.required<string>();
}
```

Avec ActivatedRoute

```ts
private route = inject(ActivatedRoute);
id = this.route.snapshot.paramMap.get('id');
```

Avec withComponentInputBinding(), les paramètres de route (:id), les query params et les données des resolvers arrivent directement dans les inputs du composant.

> ActivatedRoute reste disponible. snapshot représente la route à l'instant T. Les observables paramMap et queryParamMap suivent les changements.

Exercice bonus · Routes et paramètres

Pratiquer le concept

≈ 15 min

1. Une liste d'articles /blog et un détail /blog/:slug reçu en input (withComponentInputBinding)
2. Un query param ?page= lu en input pour paginer la liste
3. Un bouton « Article suivant » qui utilise router.navigate()

Guards

auth-guard.ts (ng g guard guards/auth)

```
import { CanActivateFn, Router } from '@angular/router';
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  return auth.isLoggedIn() ? true :
router.parseUrl('/login');
};
```

app.routes.ts

```
{ path: 'admin', component: Admin, canActivate:
[authGuard] }
```

Un guard décide si une route peut être activée. Il retourne true, false ou une redirection (UrlTree).

Type	Contrôle
canActivate	Accès à la route
canActivateChild	Accès aux routes enfants
canDeactivate	Sortie de la route
canMatch	Choix de la route (remplace canLoad)

Les guards sont des fonctions. Les guards en classe sont dépréciés.

Resolvers

fan-resolver.ts

```ts
export const fanResolver: ResolveFn<Fan> = (route) => {
  const id = Number(route.paramMap.get('id'));
  return inject(FanService).getById(id);
};
```

fans.routes.ts

```ts
{ path: ':id', component: FanDetail,
  resolve: { fan: fanResolver } }
```

fan-detail.ts (withComponentInputBinding)

```ts
fan = input.required<Fan>();
```

Un resolver charge une donnée avant l'affichage de la route. Le composant reçoit l'objet déjà chargé et évite les valeurs undefined.

Il retourne une valeur, une Promise ou un Observable.

> resource() et httpResource() offrent une alternative qui affiche la page tout de suite avec un état de chargement.

Exercice bonus · Guards et resolvers

Pratiquer le concept

≈ 15 min

1. authGuard qui redirige vers /login si un signal isLoggedIn vaut false
2. canDeactivate sur un formulaire, avec confirm() si des changements ne sont pas enregistrés
3. Un resolver qui charge l'article avant d'afficher le détail

À VOUS DE JOUER

Exercice

Compléter le CRUD de fans de séries :

- Fichier de routes fans.routes.ts chargé en lazy loading
- Paramètre :id reçu comme input dans les composants de détail et de modification
- Guard canDeactivate qui demande une confirmation si le formulaire de modification contient des changements non enregistrés
- Page 404 pour les routes inconnues

[icon: Flag]

Fil rouge ShopAdmin · Jour 4 · étape 9 / 13

Navigation et protection

- Routes /catalogue/:id (détail) et /catalogue/:id/edit, avec l'id reçu en input
- Après l'enregistrement, navigation vers la page de détail
- canDeactivate sur le formulaire de modification s'il contient des changements
- adminGuard sur la création et la modification (signal isAdmin dans un AuthService), et une page 404

Livrable

Un CRUD produit complet et protégé, avec une URL par écran.

11

Storage

Stockage dans le navigateur

Les storages

11.1 Différence entre les deux storages

11.2 Méthodes associées

Différence entre les deux storages

`sessionStorage`

garde les informations jusqu'à la fermeture de l'onglet

`localStorage`

les garde jusqu'à ce que l'application vide le storage ou que l'utilisateur efface les données du navigateur

Les deux stockent environ 5 Mo par site, uniquement sous forme de texte. Les objets sont enregistrés avec JSON.stringify() et relus avec JSON.parse().

> Le contenu est lisible par tout script de la page. Il ne faut jamais y stocker de mot de passe ou de donnée sensible. En SSR, localStorage n'existe pas côté serveur.

Méthodes associées

Méthode	Effet
setItem('key', value)	Ajoute ou remplace une valeur (string uniquement)
getItem('key')	Récupère la valeur stockée à la clé, ou null
removeItem('key')	Supprime une clé et sa valeur
clear()	Vide complètement le storage

> Les commandes sont identiques pour localStorage et sessionStorage. Seule la durée de conservation change.

Synchroniser un signal

```
@Injectable({ providedIn: 'root' })
export class CartStorage {
  items = signal<string[]>(
    JSON.parse(localStorage.getItem('cart') ?? '[]')
  );

  constructor() {
    effect(() => {
      localStorage.setItem(
        'cart', JSON.stringify(this.items()));
    });
  }
}
```

Exercice bonus · localStorage et sessionStorage

Pratiquer le concept

≈ 15 min

1. Mémoriser le thème choisi dans le localStorage et le restaurer au démarrage
2. Brouillon de formulaire en sessionStorage, effacé après l'envoi
3. Bouton « Tout effacer » avec clear(), et gestion d'un JSON invalide avec try / catch

[icon: Flag]

Fil rouge ShopAdmin · Jour 5 · étape 10 / 13

Panier persistant

```
git checkout j5-depart
```

- effect() qui sauvegarde le panier dans le localStorage à chaque changement
- Restauration du panier au démarrage de l'application
- Bouton « Valider la commande » qui vide le panier et le storage

Livrable

Le panier survit à un rechargement de la page.

12

Observables et RxJS

Flux asynchrones

Les observables

12.1 La librairie RxJS

12.2 Subject et BehaviorSubject

12.3 subscribe()

12.4 Désabonnement

12.5 Opérateurs RxJS

12.6 Signals et RxJS

La librairie RxJS

RxJS est installée avec Angular. Elle gère les Observables, des flux de valeurs asynchrones : requêtes HTTP, événements, websockets, timers.

Elle fournit des opérateurs pour transformer, filtrer, combiner et annuler ces flux. Depuis l'arrivée des signals, la répartition est la suivante :

Signals

pour l'état affiché par les composants

RxJS

pour les flux d'événements et les traitements asynchrones complexes (annulation, debounce, combinaison de sources)

Ce chapitre couvre l'essentiel. Référence : rxjs.dev et learnrxjs.io

Subject et BehaviorSubject

notification service

```
@Injectable({ providedIn: 'root' })
export class NotificationService {
  private messages = new BehaviorSubject<string[]>([]);
  messages$ = this.messages.asObservable();
  push(message: string) {
    this.messages.next(
      [...this.messages.getValue(), message]);
  }
}
```

- **Subject** émet les nouvelles valeurs à ses abonnés
- **BehaviorSubject** a une valeur initiale, garde la dernière valeur (getValue()) et l'envoie à chaque nouvel abonné
- **next()** émet une nouvelle valeur
- **asObservable()** expose le flux sans permettre next() depuis l'extérieur

> Pour un état partagé simple, un signal dans un service est aujourd'hui plus simple (chapitre 8).

subscribe()

TypeScript

```typescript
this.notifications.messages$.subscribe({
  next: (value) => console.log(value),
  error: (err) => console.error(err),
  complete: () => console.log('Terminé'),
});
```

Pipe async dans le template

```
@for (m of messages$ | async; track $index) {
  <p>{{ m }}</p>
}
```

subscribe() reçoit un objet avec trois fonctions :

- **next** : traitement de chaque valeur émise
- **error** : traitement en cas d'erreur
- **complete** : fin du flux

> Un Observable ne fait rien tant que personne n'y est abonné.

Le pipe async s'abonne et se désabonne automatiquement.

Désabonnement

takeUntilDestroyed()

```
export class Notifications {
  private service = inject(NotificationService);

  constructor() {
    this.service.messages$
      .pipe(takeUntilDestroyed())
      .subscribe((m) ⇒ console.log(m));
  }
}
```

Ancienne méthode

```
private sub?: Subscription;
ngOnDestroy() { this.sub?.unsubscribe(); }
```

> Un abonnement non fermé continue après la destruction du composant et provoque des fuites mémoire.

Solutions :

- **takeUntilDestroyed()** ferme l'abonnement à la destruction
- Le pipe **async** dans le template
- **toSignal()** (12.6)
- **unsubscribe()** dans ngOnDestroy (projets existants)

Opérateurs RxJS

Recherche avec suggestions

```
this.searchTerms$
  .pipe(
    debounceTime(300),
    distinctUntilChanged(),
    switchMap((term) => this.api.search(term)),
    map((results) => results.slice(0, 10)),
    catchError(() => of([])),
  )
  .subscribe((results) => this.results.set(results));
```

Opérateur	Effet
map	Transforme chaque valeur
filter	Ne garde que certaines valeurs
debounceTime	Attend une pause dans le flux
distinctUntilChanged	Ignore les doublons consécutifs
switchMap	Annule la requête précédente
catchError	Gère une erreur

Exercice bonus · RxJS

Pratiquer le concept

≈ 15 min

1. interval(1000) affiché avec le pipe async
2. fromEvent sur un input, avec debounceTime et distinctUntilChanged, pour loguer la recherche
3. Un BehaviorSubject compteur dans un service, avec deux composants abonnés
4. Convertir le flux en signal avec toSignal()

Signals et RxJS

TypeScript

```ts
import { toSignal, toObservable }
  from '@angular/core/rxjs-interop';

export class Notifications {
  private service = inject(NotificationService);

  messages = toSignal(this.service.messages$,
    { initialValue: [] });
  query = signal('');
  query$ = toObservable(this.query);
}
```

- **toSignal()** convertit un Observable en signal et gère l'abonnement
- **toObservable()** convertit un signal en Observable pour utiliser les opérateurs RxJS
- **rxResource()** équivaut à resource() avec un loader qui retourne un Observable

[icon: Flag]

Fil rouge ShopAdmin · Jour 5 · étape 11 / 13

Recherche dans le catalogue

- Champ de recherche au-dessus du catalogue
- toObservable() sur le texte saisi, puis debounceTime(300) et distinctUntilChanged()
- Résultat filtré récupéré avec toSignal() et affiché dans la grille

Livrable

Une recherche qui ne se déclenche qu'après une pause de frappe.

13

HttpClient

Communiquer avec une API

HttpClient

13.1 API et Angular

13.2 Verbes HTTP

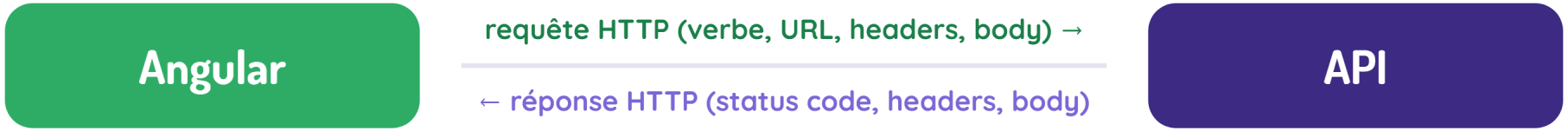
13.3 Service d'accès à l'API

13.4 httpResource()

13.5 Interceptor

API et Angular

La communication avec une API passe par le service HttpClient :



Chaque méthode de HttpClient retourne un Observable<T>. La requête n'est envoyée qu'au moment du subscribe().

Depuis	Changement
Angular 21	HttpClient est fourni d'office. provideHttpClient() ne sert plus qu'à ajouter des options (interceptors)
Angular 22	Le backend fetch est utilisé par défaut. withFetch() n'est plus nécessaire

app.config.ts

```
provideHttpClient(withInterceptors([authInterceptor]))
```

Verbes HTTP

Verbe	Utilité	Exemple Angular
GET	Récupération de données	this.http.get<Fan[]>(this.url)
POST	Création (une réponse peut être reçue)	this.http.post<Fan>(this.url, fan)
PUT	Remplacement d'une ressource	this.http.put<Fan>(`${this.url}/${id}`, fan)
PATCH	Modification partielle	this.http.patch(`${this.url}/${id}`, changes)
DELETE	Suppression	this.http.delete(`${this.url}/${id}`)

- Le type entre chevrons (get<T>) décrit la réponse attendue
- Un paramètre optionnel ajoute des headers ou des query params : { headers: { Authorization: '...' }, params: { page: 2 } }

Service d'accès à l'API

fan service

```ts
@Injectable({ providedIn: 'root' })
export class FanService {
  private http = inject(HttpClient);
  private url = 'http://localhost:3000/fans';

  getAll() {
    return this.http.get<Fan[]>(this.url);
  }

  create(fan: Omit<Fan, 'id'>) {
    return this.http.post<Fan>(this.url, fan);
  }

  delete(id: number) {
    return this.http.delete<void>(`${this.url}/${id}`);
  }
}
```

Les appels HTTP sont regroupés dans un service, un service par ressource de l'API.

Le composant appelle le service et s'abonne au résultat (subscribe(), pipe async ou toSignal()).

Dans le composant

```ts
this.fanService.create(fan)
  .subscribe(() =>
    this.router.navigate(['/fans']));
```

Exercice bonus · HttpClient

Pratiquer le concept

≈ 20 min

1. Lancer json-server avec une ressource /todos
2. TodoService : getAll, create, update (PATCH du champ done) et delete
3. Liste avec ajout, case à cocher et suppression, rafraîchie après chaque écriture

httpResource()

TypeScript

```ts
export class FanList {
  fans = httpResource<Fan[]>(
    () => 'http://localhost:3000/fans');
}

export class FanDetail {
  id = input.required<string>();
  fan = httpResource<Fan>(
    () => `http://localhost:3000/fans/${this.id()}`);
}
```

fan-list.html

```html
@if (fans.isLoading()) { <p>Chargement...</p> }
@if (fans.error()) { <p>Erreur de chargement</p> }
@for (fan of fans.value() ?? []; track fan.id) {
  <p>{{ fan.name }}</p>
}
```

httpResource() (stable depuis Angular 22) lance une requête GET et expose le résultat sous forme de signals.

Quand un signal utilisé dans l'URL change, la requête est relancée. reload() la relance à la demande.

> Pour les écritures (POST, PUT, DELETE), HttpClient reste utilisé.

Interceptor

auth-interceptor.ts

```ts
import { HttpInterceptorFn } from '@angular/common/http';
export const authInterceptor: HttpInterceptorFn =
  (req, next) => {
    const token = inject(AuthService).token();
    if (!token) {
      return next(req);
    }
    const authReq = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` },
    });
    return next(authReq);
  };
```

app.config.ts

```ts
provideHttpClient(withInterceptors([authInterceptor]))
```

Un interceptor intercepte chaque requête HTTP pour la modifier : ajout d'un header, gestion des erreurs, loader.

- La requête est immuable. **clone()** crée une copie modifiée
- **next()** transmet la requête à la suite de la chaîne
- Les interceptors fonctionnels remplacent les classes HttpInterceptor et HTTP_INTERCEPTORS

Exercice bonus · httpResource() et interceptors

Pratiquer le concept

≈ 15 min

1. Remplacer getAll par httpResource(), avec les états chargement et erreur
2. Interceptor qui mesure la durée de chaque requête et la logue
3. Interceptor qui ajoute le header Accept-Language: fr-BE

À VOUS DE JOUER

Exercice

Connecter le CRUD de fans de séries à une API REST (json-server) :

- Un service FanService regroupe les appels HTTP
- La liste et le détail utilisent httpResource()
- La création, la modification et la suppression utilisent HttpClient
- Un interceptor ajoute un header X-App: fans à chaque requête et affiche les erreurs HTTP dans la console

[icon: Flag]

Fil rouge ShopAdmin · Jour 5 · étape 12 / 13

API, clients et commandes

- API avec json-server ou l'API .NET : /products, /customers et /customers/:id/orders paginé
- Catalogue chargé avec httpResource(), création, modification et suppression avec HttpClient
- Interceptor : header X-App: shop-admin et log des erreurs HTTP
- Page /clients : tableau clients avec sous-tableau de commandes paginé (CustomersPage et CustomerOrders smart, CustomerTable et OrderTable dumb)

Livrable

Toute l'application tourne sur une vraie API.

14

Publication et mises à jour

Build et ng update

Build de l'application

La commande ng build compile l'application en mode production (par défaut).

Le résultat est placé dans dist/<nom-du-projet>/browser. Ce dossier se dépose sur n'importe quel serveur web (Apache, Nginx, IIS, hébergement statique).

L'option --prod n'existe plus. --configuration development produit un build de développement.

terminal

```
ng build
ng build --base-href /mon-app/
```

Si l'application n'est pas déposée à la racine du serveur, préciser le chemin avec --base-href. Il est inscrit dans la balise <base href="/"> de index.html.

Le serveur doit renvoyer index.html pour toutes les URL de l'application. Sans cette règle, un rafraîchissement sur /fans/42 donne une erreur 404.

Mettre à jour Angular

Une version majeure d'Angular sort tous les 6 mois. Chaque version est supportée pendant 18 mois.

ng update affiche les mises à jour disponibles et lance les migrations automatiques du code.

> Il est conseillé de monter une version majeure à la fois et de suivre le guide angular.dev/update-guide.

terminal

```
ng update
ng update @angular/core@22 @angular/cli@22
```

Migration (ng generate @angular/core:...)	Effet
standalone	NgModules => composants standalone
control-flow	*ngIf, *ngFor => @if, @for
inject	Constructeur => inject()
signal-input-migration	@Input() => input()
output-migration	@Output() => output()
route-lazy-loading	Routes => lazy loading

Exercice bonus · Build et mises à jour

Pratiquer le concept

≈ 15 min

1. ng build, puis servir dist/<projet>/browser avec npx http-server et tester un refresh sur une route
2. Refaire un build avec --base-href /demo/ et comparer la balise <base> de index.html
3. Lancer ng update pour lister les mises à jour disponibles, sans les appliquer

[icon: Flag]

Fil rouge ShopAdmin · Jour 5 · étape 13 / 13

Livraison

- ng build de ShopAdmin
- Servir le dossier dist et tester un rafraîchissement sur /catalogue/3
- Démo finale : chaque stagiaire présente son ShopAdmin en 5 minutes

Livrable

Une application buildée, prête à être déployée.

bstorm TRAINING

Merci pour votre attention.

Indice · Exercice bonus Composants (slide 26)

Un coup de pouce

Bloqué depuis 5 min ?

1. La génération se fait avec une commande du CLI, une fois par composant. — Slide 22
2. Un composant enfant ne s'affiche que si son parent le connaît. Il y a deux endroits à modifier dans le parent. — Slide 24
3. Pour le template inline, une autre propriété du décorateur remplace celle qui pointe vers un fichier. Elle attend le HTML lui-même. — Slide 23
4. Ce qu'on affiche avec {{ }} doit d'abord exister dans la classe. — Slide 25

Indice · Exercice bonus Cycle de vie et navigation (slide 34)

Un coup de pouce

Bloqué depuis 5 min ?

1. Les liens ne réagissent pas ? Regardez ce qu'importe le composant qui contient les liens. Slide 29
2. Pour loguer, la classe implémente les interfaces OnInit et OnDestroy. Slide 28
3. loadChildren charge un fichier de routes. C'est dans ce fichier que se déclare la route enfant. Slides 30-31
4. La route enfant a besoin d'un endroit où s'afficher dans Page A. Slide 30

Indice 1 / 2 · Exercice Home et Exercices (slide 35)

Par où commencer

Bloqué depuis 10 min ?

1. Créez d'abord toute la structure (composants et fichier de routes de la fonctionnalité), le code ensuite. — Slide 22
2. Les routes principales ne contiennent que les points d'entrée : la redirection et les deux fonctionnalités. — Slides 29, 31
3. Le nom écrit après m. dans loadChildren doit correspondre exactement à ce que le fichier de routes exporte. — Slide 31
4. Exercices joue le rôle de parent : ses trois exercices sont ses enfants. — Slide 30
5. Testez chaque URL à la main dans la barre d'adresse avant de construire le menu.

Indice 2 / 2 · Exercice Home et Exercices (slide 35)

Les pièges fréquents

Bloqué depuis 15 min ?

1. Une URL enfant affiche une page vide ? Les chemins s'écrivent par rapport au parent. — Slide 32
2. Les liens ne réagissent pas ? C'est le composant qui contient les liens qui doit importer ce qu'il faut. — Slide 29
3. Un composant ne s'affiche pas ? Son parent doit le déclarer avant de l'utiliser. — Slide 24
4. Le titre de Exercices s'affiche mais jamais l'exercice ? Le parent doit leur réserver une place. — Slide 30
5. Page blanche ? La console du navigateur (F12) indique presque toujours le fichier en cause.